Lecture 11: Kernel III — Memory Allocator Design

Session 13 · Mon Oct 12, 2026

Recap: Lectures 9 and 10

Lecture 9. A program asks the kernel for anything it cannot do alone through a **system call**, and a system call costs as much as hundreds of function calls. So do more work per call.

Lecture 10. Each process has its own **virtual address space**. `mmap` attaches a new range of addresses to it, and the physical pages behind them arrive on first touch, through page faults.

Today. `malloc` sits between the two:

```
program  --- malloc / free: function calls, millions of them --->  allocator
allocator --- sbrk / mmap: system calls, a few big ones -------->  kernel
```

The allocator gets large ranges from the kernel rarely, and hands out small pieces of them often, without a system call each time.

Outline

What you will learn today:

1. What an allocator does, and the rules it works under
2. How a block is laid out: header, payload and footer
3. How **`malloc`** finds room: the implicit free list and first fit
4. Splitting and coalescing, and why every block has a footer
5. Faster searching with an explicit free list
6. How to debug an allocator with a heap checker
7. What Lab 3 asks you to build, level by level

What an Allocator Does

`malloc(n)` hands the program `n` bytes, and `free(p)` gives them back. Under both is one large region of memory, the **heap**. The allocator gets it from the kernel with `mmap` (Lecture 10) or with `sbrk(n)`, a system call that moves the end of the heap up by `n` bytes and returns the old end. The allocator cuts the heap into **blocks** and keeps track of which are in use.

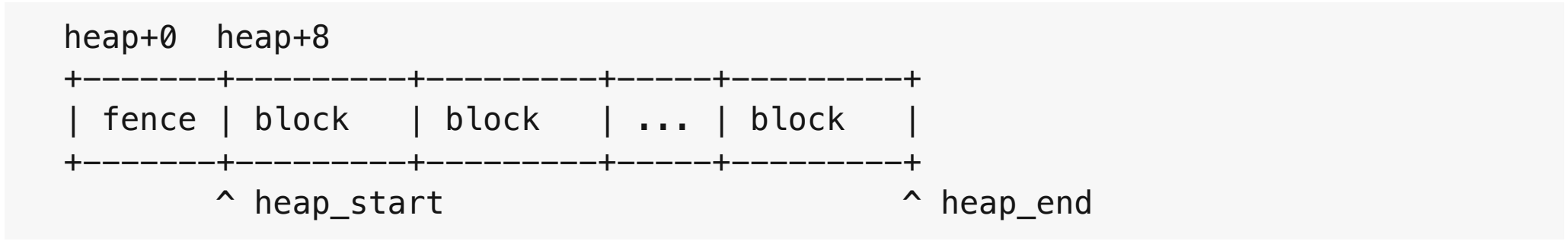
It works under three rules:

- It cannot move a block after handing it out, because the program holds pointers into it.
- It must answer each request at once, without knowing what comes next.
- Every address it returns must be a multiple of 16, so any type fits there.

It is judged on **speed**, the time per request, and on **utilization**, the share of the heap that holds data the program actually asked for.

In Lab 3, which goes out today, you write one. Every slide from here on uses Lab 3's names, so the code you see is the code you will write.

The Heap Is a Row of Blocks



The blocks sit end to end, with no gaps, from **`heap_start`** to **`heap_end`**. When no block can serve a request, the allocator asks for more memory with **`mem_sbrk(n)`**. That is Lab 3's version of **`sbrk`**: it adds **`n`** bytes at the end of the heap and returns their address.

The 8-byte **fence** before the first block is there to make coalescing safe. The coalescing slides explain it.

One Block: Header, Payload, Footer

```
+----------+-----------------------------------+----------+
|  Header  |  payload (what malloc returns)    |  Footer  |
|  8 bytes |  starts at a multiple of 16       |  8 bytes |
+----------+-----------------------------------+----------+
^ h        ^ payload_of(h)                     ^ footer_of(h)
|<---------------------- h->size ---------------------->|
```

```c
typedef struct header {
    uint32_t size;      // the whole block: header + payload + footer
    uint32_t is_free;   // 1 if free, 0 if allocated
} Header;

typedef Header Footer;  // the footer is a copy of the header
```

The **header** says how big the block is and whether it is free. The **payload** is the part the program uses. The **footer** repeats the header at the other end of the block, so that the *next* block can find this one. Each block pays 16 bytes for its header and footer.

How Big Is a Block?

A request for **`n`** bytes needs room for the payload plus 16 bytes of header and footer. That total is rounded up to a multiple of 16, so the next payload also starts on a multiple of 16, and it is never less than 32 bytes (**`MIN_BLOCK`**):

Request **`n`**	**`n`** + 16	Block size
1	17	32
16	32	32
17	33	48
48	64	64
100	116	128

In Lab 3 this rule is the function **`block_size(n)`**, the first one you write.

The bytes a block has beyond what was asked for are **internal fragmentation**. A 1-byte request uses a 32-byte block, so 31 bytes are wasted inside it.

Walking the Heap

Every block records its own size, so the blocks form a list without any pointers: start at **`heap_start`**, add the size, and you are at the next block. Lab 3 gives you one-line helpers for these steps:

```
payload_of(h)       // h + 1: the payload starts right after the header
header_of(p)        // (Header *)p - 1: from a payload back to its header
next_block(h)       // (char *)h + h->size: the block after h
at_end(h)           // h has reached heap_end, so there is no block here
set_block(h, size, is_free)   // writes the header AND the footer
```

This list of every block, free or allocated, is an **implicit free list**. “Implicit” because no pointers link it: the sizes do.

Change a block only through **`set_block`**. If a header and its footer ever disagree, the next block that looks backwards reads the wrong size.

An Implicit Free List

To find room for a block of **`size`** bytes, walk the blocks from the start and take the first free one that is big enough. This is **first fit**:

```
static Header *find_fit(size_t size)
{
    for (Header *h = heap_start; !at_end(h); h = next_block(h))
        if (h->is_free && h->size >= size)
            return h;
    return NULL;                        // nothing fits
}
```

`mm_malloc` in Lab 3 is given. It is four steps:

```
size_t size = block_size(n);
Header *h = find_fit(size);
if (h == NULL) h = grow_heap(size);   // mem_sbrk more: one new free block
if (h == NULL) return NULL;           // mem_sbrk failed, returned (void *)-1
place(h, size);                       // mark h allocated (Level 2: split it)
return payload_of(h);
```

free()

Freeing a block is a single change to its header and footer:

```
void mm_free(void *p)
{
    if (p == NULL)
        return;
    Header *h = header_of(p);
    set_block(h, h->size, 1);      // the block is now free
    coalesce(h);                   // Level 2: merge with free neighbours
}
```

Nothing is given back to the kernel. The freed block stays in the heap, marked free, and **`find_fit`** can hand it out again. This is why a program's memory use, as the operating system reports it, rarely goes down after **`free`**.

Run It: Lab 3 on the Lecture's Heap

Lab 3's first trace makes four requests, frees two blocks, and asks again. The driver runs a trace against your allocator and checks every answer. With a working Level 1 allocator:

```
$ ./mdriver -v traces/t1-lecture.tr
    7   a 0 16                  -> heap+16
    8   a 1 48                  -> heap+48
    9   a 2 16                  -> heap+112
   10   a 3 80                  -> heap+144
   16   f 1
   17   f 3
   18   a 4 112                 -> heap+240
   24   a 5 40                  -> heap+48
ok:  8 requests, peak 184 bytes in use, heap 360 bytes, utilization 51.1%
```

`a 1 48` means "request 1 asks for 48 bytes", and **`f 1`** frees it. The addresses are offsets from the start of the heap, and the trace's heap-size checks are left out here. Request 5 reuses the block request 1 gave back: **`find_fit`** found it. Request 4 did not fit anywhere, and the next slide shows why. Add **`-d`** to print the whole heap after every request.

Fragmentation

After **`f 1`** and **`f 3`**, the heap is:

```
heap+8      alloc     32 bytes
heap+40     free      64 bytes
heap+104    alloc     32 bytes
heap+136    free      96 bytes
```

Request 4 asks for 112 bytes, which needs a 128-byte block. The heap has 160 free bytes, but the largest free block is 96, so **`find_fit`** fails and the heap grows by 128 bytes. This is **external fragmentation**: there is enough free space in total, but not in one piece.

Internal fragmentation is waste inside a block. External fragmentation is waste between blocks. Splitting fights the first, and coalescing fights the second.

Splitting a Free Block

When the free block found is bigger than needed, **`place`** can cut it in two. Split only if the leftover is at least **`MIN_BLOCK`**, because a smaller piece cannot be a block. Request a 16-byte block (a 32-byte block) from the heap above. First fit picks the free 64-byte block at heap+40:

```
before                                  after a 16-byte request (Level 2)
heap+40     free      64 bytes          heap+40     alloc     32 bytes
                                        heap+72     free      32 bytes
```

The request gets 32 bytes, and the other 32 stay free for later. Without splitting, the whole 64-byte block goes to a 16-byte request and 48 bytes sit unused inside it.

Coalescing

When a block is freed next to a free block, merge them into one, so a large request can use the space later. Merging with the **next** block uses only what you have already seen:

```
Header *next = next_block(h);
if (!at_end(next) && next->is_free)          // is there a next block?
    set_block(h, h->size + next->size, 1);   // one bigger free block
```

Two details matter:

- The last block has no next block. Past **`heap_end`** there is only garbage, so check **`at_end`** before reading **`next->is_free`**.
- **`set_block`** writes the new size into the header *and* into the footer at the far end. The old footer and header in the middle are now just bytes inside the merged block.

Coalescing: The Four Cases

The block before **`h`** may be free too. A freed block has four possible situations:

Block before	Block after	Result
allocated	allocated	**`h`** alone
allocated	free	**`h`** + next
free	allocated	previous + **`h`**
free	free	previous + **`h`** + next

Going forward is easy: **`next_block(h)`**. Going back needs the footer. The 8 bytes just before **`h`** are the previous block's footer, **`prev_footer(h)`**. It says whether that block is free and how big it is, so **`prev_block(h)`** is **`h`** minus that size. A size stored at both ends of a block is called a **boundary tag**.

For the first block, the 8 bytes before it are the fence. The fence reads as "allocated, size 0", so the first block never tries to merge backwards.

Run It: Coalescing Undoes the Fragmentation

Take the fragmented heap again and also free the 32-byte block in the middle. Both of its neighbours are free, so this is the fourth case:

```
before f 2                              after f 2 (Level 2)
heap+8      alloc     32 bytes          heap+8      alloc     32 bytes
heap+40     free      64 bytes          heap+40     free     192 bytes
heap+104    alloc     32 bytes
heap+136    free      96 bytes
```

Now the 112-byte request, which needs 128 bytes, fits at heap+40, and the leftover 64 bytes are split off as a free block:

Allocator	Where the 128-byte block goes	Heap size
Level 1: no coalescing	at the end, heap+240	360 bytes
Level 2: split and coalesce	heap+48, inside the merged block	232 bytes

Allocation Strategies

Strategy	Takes	Speed	Waste
First fit	the first block big enough	fast, stops early	small blocks pile up near the start
Next fit	the first block big enough, starting where the last search stopped	faster still	usually more fragmentation
Best fit	the smallest block big enough	must look at every free block	least waste per request

Lab 3 uses first fit. Its bonus asks you to beat first fit on utilization.

Real allocators such as glibc's `malloc` and jemalloc keep separate lists for different **size classes**, so finding a block of the right size takes one step instead of a search.

An Explicit Free List

`find_fit` on an implicit list walks past every allocated block to reach the free ones. An **explicit free list** links only the free blocks, through two pointers stored in each free block's unused payload:

```
free block:   | Header | next | prev | ...unused... | Footer |
                        8 B    8 B
```

`find_fit` follows **`next`** and never sees an allocated block. A block joins the list when freed and leaves it when allocated or merged. This is why **`MIN_BLOCK`** is 32: a header, two pointers and a footer.

Lab 3's trace **`t10-many`** keeps 30,000 blocks allocated while it frees and requests 30,000 more. On an Apple M1 Pro, the implicit list (Level 2) takes **1.7 s** and the explicit list (Level 4) takes **0.001 s**, building exactly the same heap.

Debugging Your Allocator: A Heap Checker

AddressSanitizer and Valgrind check programs that *use* **`malloc`**. They cannot check the inside of an allocator: to them, your heap is one big region of memory that you own, and any byte of it is fine to touch.

So an allocator checks itself. A **heap checker** walks every block and reports anything impossible:

- a block size that is not a multiple of 16, or is below **`MIN_BLOCK`**
- a header that does not match its footer
- two free blocks next to each other (a missed coalesce)
- a walk that does not end exactly at **`heap_end`**
- a free block missing from the free list, or an allocated block on it

In Lab 3 this is **`mm_check`**, Level 5. **`./mdriver -c`** runs it after every request, so it stops at the first request that broke the heap. Write it early: it finds bugs in every other level.

Lab 3 at a Glance

Level	Points	You write	Slides
1	60	**`block_size`**, **`find_fit`**, **`grow_heap`**, **`place`**, one line of **`mm_free`**	Block Size, An Implicit Free List, free()
2	15	splitting in **`place`**, **`coalesce`**	Splitting, Coalescing, The Four Cases
3	10	**`mm_realloc`**, **`mm_calloc`**	the handout
4	10	an explicit free list	An Explicit Free List
5	5	**`mm_check`**, and no compiler warnings	A Heap Checker

Level 1 alone is a passing grade. The checkpoint is Mon Oct 26: show **`make test L=1`** passing its first check, the trace on the Run It slide.

Draw the heap on paper before you code, and run **`./mdriver -v -d`** on a small trace when a check fails. Most allocator bugs are a size that is off by 8 or 16 bytes.

AI Systems Connection

Frameworks do not call the GPU allocator for each tensor. They allocate a few enormous blocks up front and sub-allocate out of them, with a free list, size classes, splitting and coalescing. That is the allocator from this lecture, in user space, for GPU memory.

They do it because the GPU allocator is slow: it can make the GPU finish its outstanding work before it returns. A training step that allocates and frees thousands of intermediate tensors would spend much of its time stopped. A caching allocator turns almost all of those requests into taking a block off a free list.

The cost is this lecture's failure mode. Out-of-memory errors during training often arrive while plenty of memory is free, just not in one piece. Inputs of varying lengths produce blocks of many sizes, those blocks get split and do not merge back, and eventually nothing can serve a large request. That is external fragmentation, and the usual advice, padding to a few fixed sizes and keeping allocation patterns regular, is the advice you would give any allocator.

The Lecture in One Slide

- An allocator cuts one heap into blocks. It cannot move a block, and it must answer each request at once.
- Each block is a header (size and free bit), a payload, and a footer that copies the header. Block sizes are multiples of 16.
- An implicit free list walks every block by its size. First fit takes the first free block that is big enough.
- Splitting reduces waste inside blocks. Coalescing reduces waste between them, and the footer lets a freed block find the block before it.
- An explicit free list links only the free blocks, through pointers stored in their payloads.
- An allocator needs its own heap checker, because sanitizers cannot see inside it.

Practice

1. What block size does each request need: 8, 24, 32 and 100 bytes?
2. The heap is the one on the Fragmentation slide. With first fit and splitting, where does a request for 40 bytes go? A request for 70 bytes?
3. In that heap, the program frees the allocated block at heap+104. Which blocks are left?
4. A block can find the next block without a footer. Why can it not find the previous block without one?
5. Why is the smallest block 32 bytes once you use an explicit free list?
6. What happens if a program frees the same pointer twice?

Answers to Practice

1. 8 needs 32 (8 + 16 = 24, raised to **`MIN_BLOCK`**). 24 needs 48 (40 rounded up). 32 needs 48. 100 needs 128 (116 rounded up).
2. 40 bytes needs a 64-byte block. The free block at heap+40 is exactly 64, so it is used whole and the payload is heap+48. 70 bytes needs 96 (86 rounded up). The 64-byte block is too small, so first fit takes the 96-byte block at heap+136, and the payload is heap+144.
3. Both of its neighbours are free, so all three merge: an allocated 32-byte block at heap+8, then one free 192-byte block at heap+40.
4. The header at the start of a block holds its size, so the next block is that many bytes ahead. To step back you need the previous block's size, and without a footer that size is only at the previous block's *start*, which you cannot find without knowing the size.
5. A free block must hold two 8-byte pointers in its payload, plus the 8-byte header and the 8-byte footer: 32 bytes.
6. The block may already have been merged with a neighbour or handed out again, so the second **`free`** corrupts the heap or the free list, and two later **`malloc`** calls can return the same memory. glibc notices some cases and stops the program with **`free(): double free detected in tcache 2`**; macOS also stops it. Your Lab 3 allocator does not check.

Reading

- CS:APP §9.9 (Dynamic Memory Allocation)
- The Lab 3 handout
- Malloc tutorial